



Classes and Objects

Discussion Flow



Object-oriented Programming



Objects



Class



Attributes



Class Creation



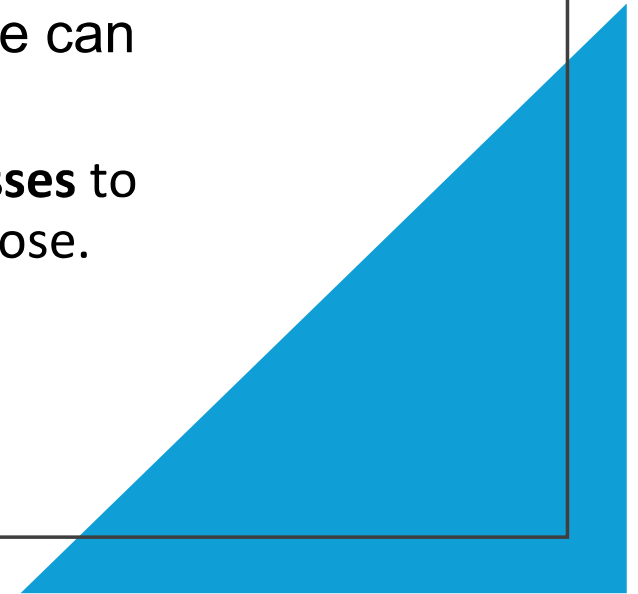
Class Instantiation



Accessing Attributes

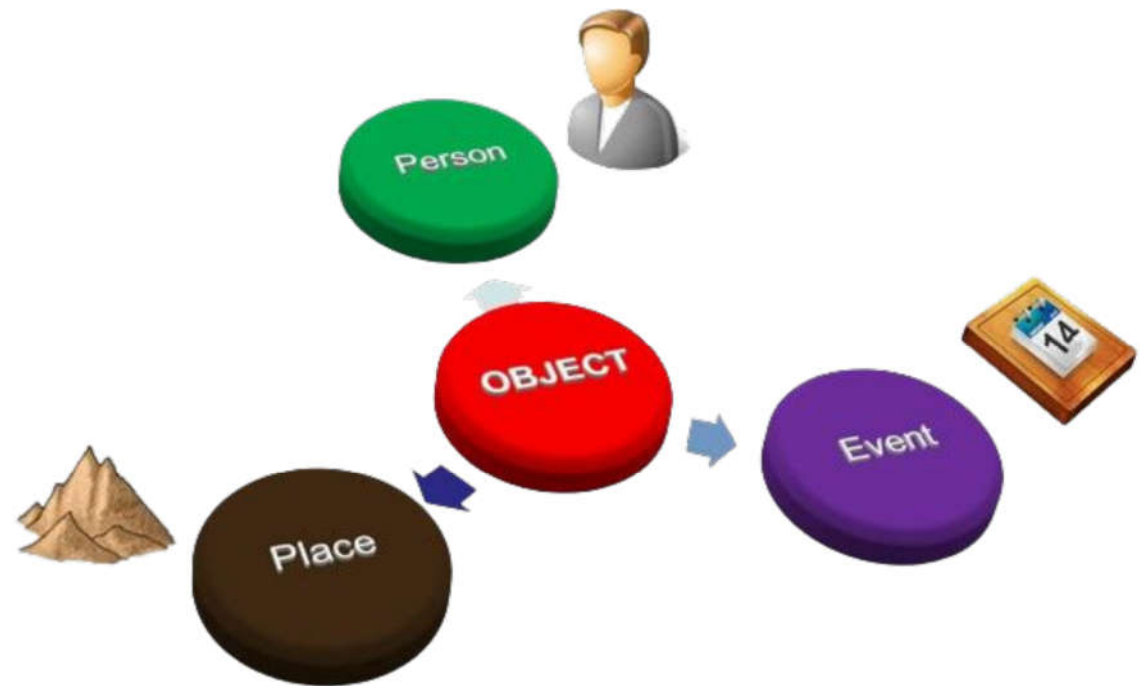
Object Oriented Programming

- It aims to implement real world objects / entities its attributes and behavior into programming so that we can represent it in our games, software or application
- It focuses on implementing real word objects using **Classes** to create variation of **Objects** that has attributes and purpose.
- It helps us create much flexible and efficient code that procedural programming



What is an object?

- An **object** is a computer representation of some real-world thing like: (person, place or event).

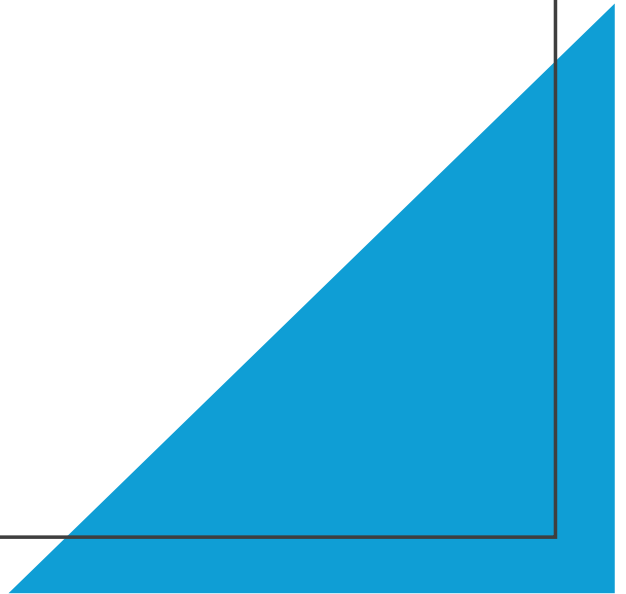


Objects

- Anything that has an attribute and a purpose is an object.

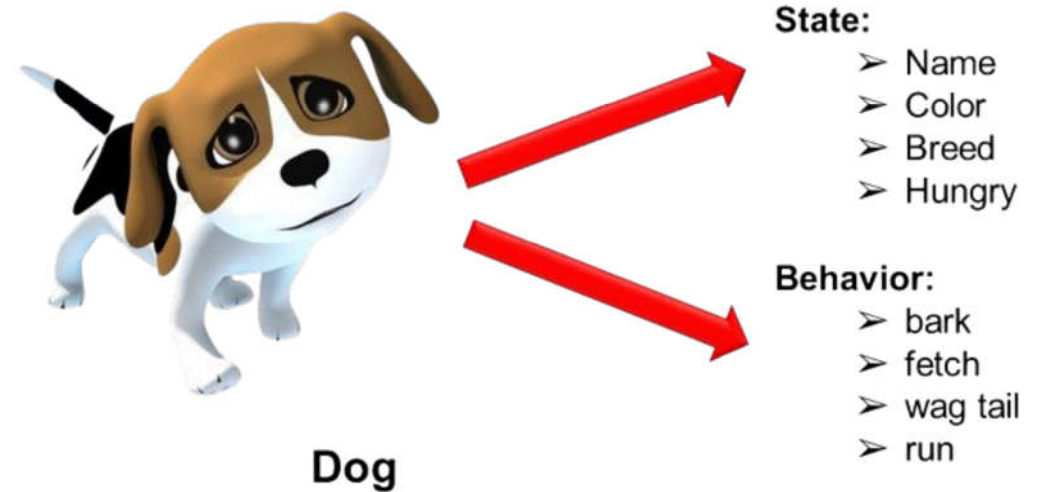
Example

- Game Characters
- Real Life People (Students, Employees...)
- Real Life Objects (Fruit, Table...)



Two features of Objects

- **State**
 - Data associated with an
 - object.
- **Behavior**
 - An action that an object is
 - capable of performing.



Exercises 1

- Classify the given words into the state and behavior of the given object.

State

Behavior



radio

on

increase volume

tune

current station

turn on

scan

decrease volume

off

seek

turn off

current volume

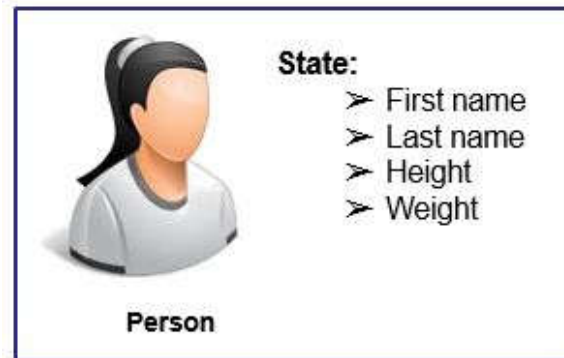
Abstract Objects and Instances

- **Abstract Object**

- A description of a real object minus details

- **Real Object**

- An instance of the abstract object



The abstract person is a model for a real person.



Class

- A class is a kind of data type, just like a string, integer or list. When we create an object of that data type, we call it an instance of a class
 - The data values which we store inside an object are called attributes, and the functions which are associated with the object are called methods. We have already used the methods of some built-in objects, like strings and lists.
-

Class

- Used in Programming to make a blueprint for our object. It will represent their attribute and purpose.

Example

Game Characters

Purpose

Physical Attack

Magic Attack

Attributes

Name

HP

SP

ATK

LVL

Example

Product

Purpose

Restock

Adjust

Delete

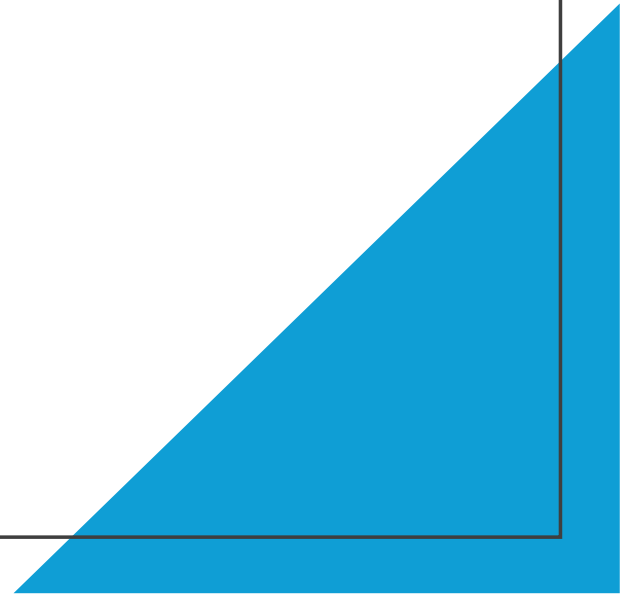
Attributes

ID

Name

Price

Quantity

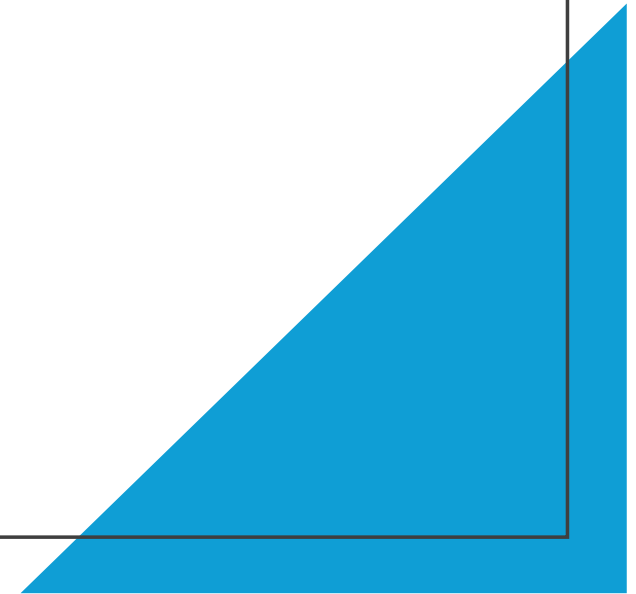


Creating Classes

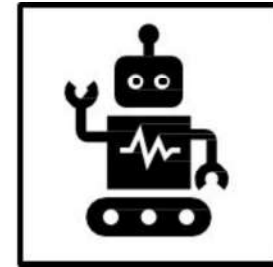
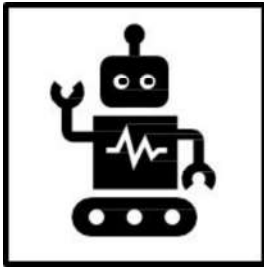
Example

Syntax

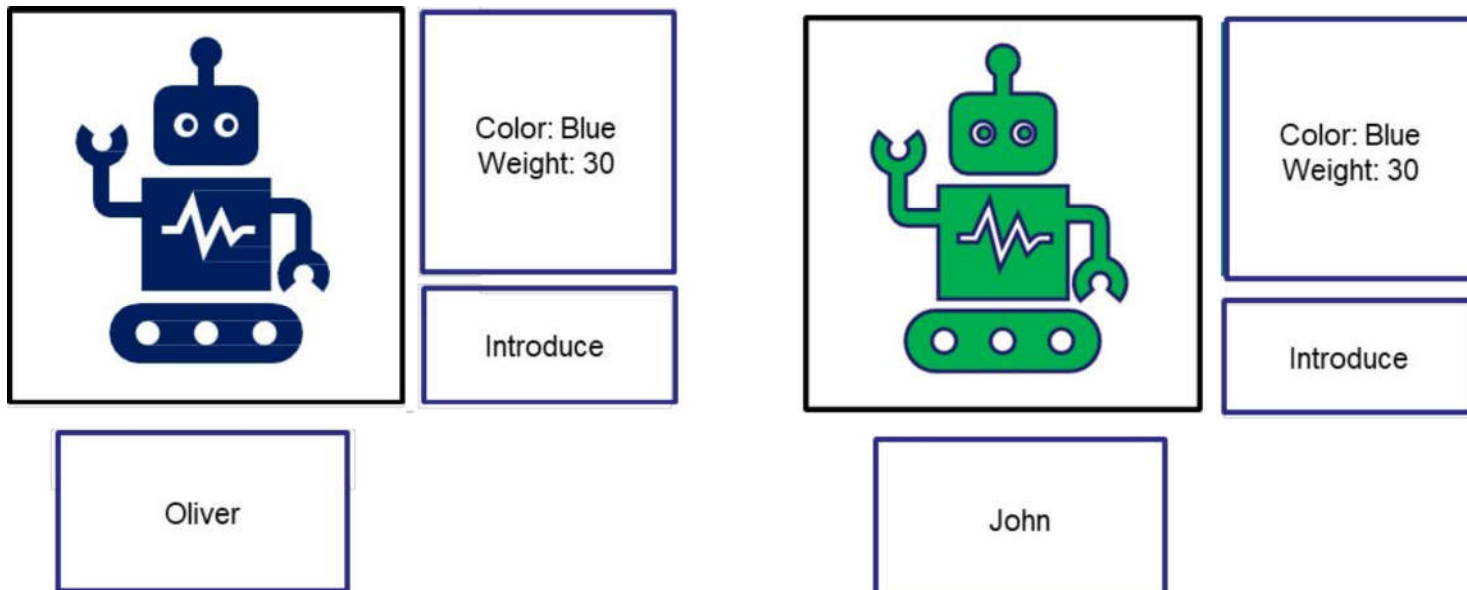
```
class className:      class Character
    #purpose / functions    name = "Name"
    #attributes              hp = 100
                             sp = 100
                             atk = 12
                             lvl = 2
```



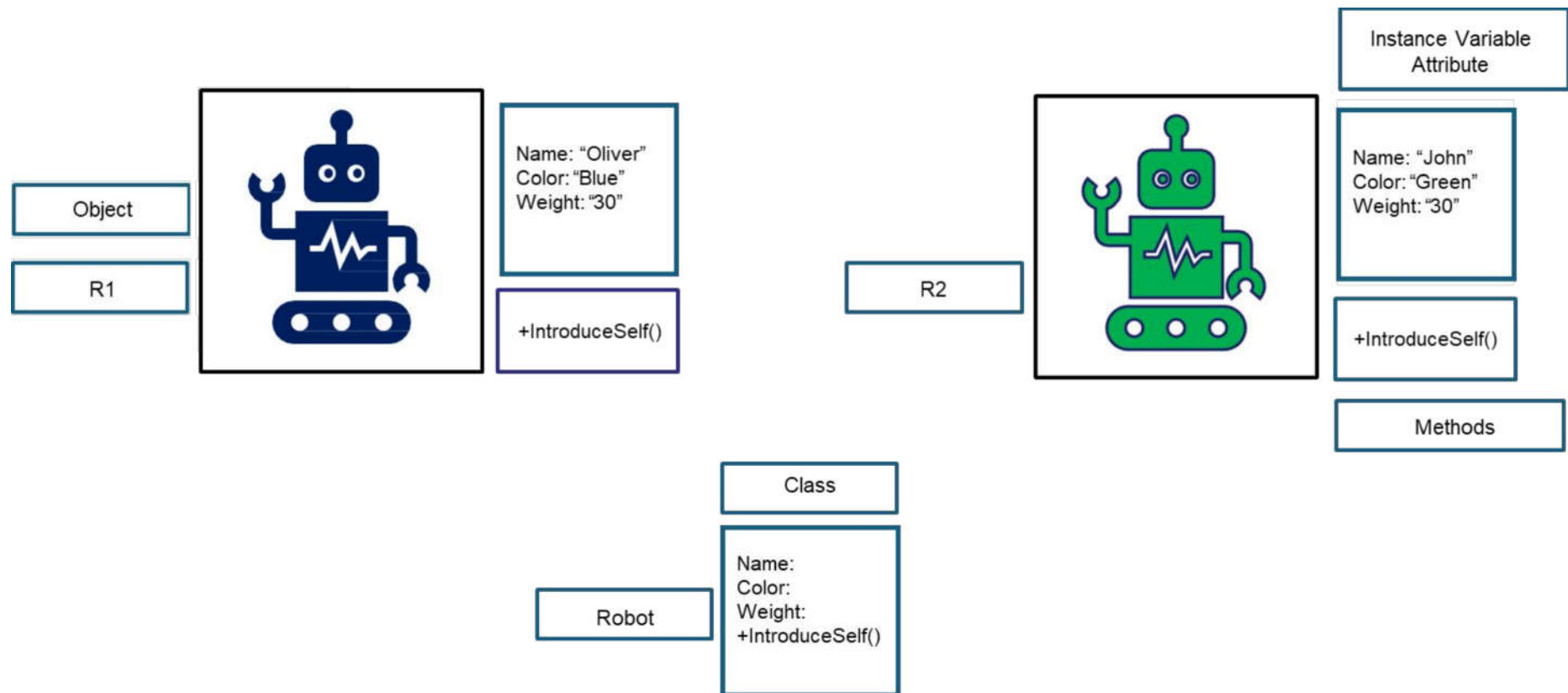
System



System

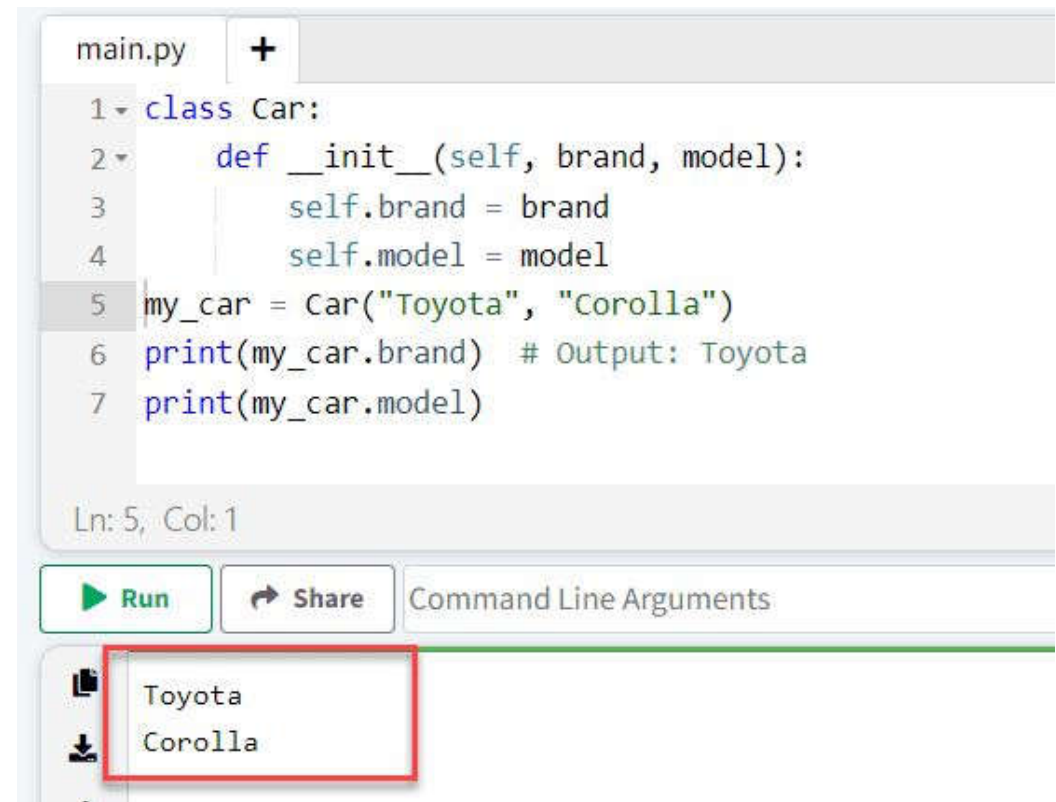


System



__init__

- `__init__()` is an instance method that initializes a newly created object.
- It takes the object as its first argument followed by additional arguments.



```
main.py +
1 class Car:
2     def __init__(self, brand, model):
3         self.brand = brand
4         self.model = model
5 my_car = Car("Toyota", "Corolla")
6 print(my_car.brand) # Output: Toyota
7 print(my_car.model)
```

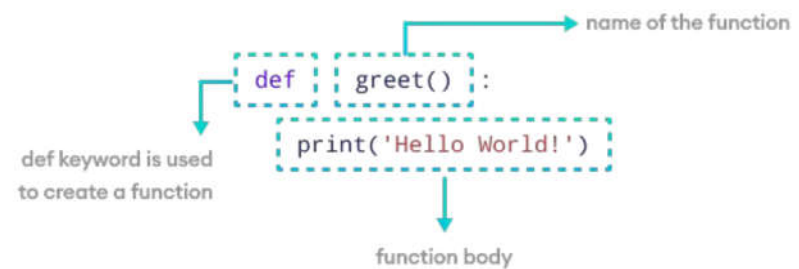
Ln: 5, Col: 1

 Run  Share Command Line Arguments

 Toyota
 Corolla

def

- define a function with the def keyword, then **write the function identifier (name) followed by parentheses and a colon.**



self

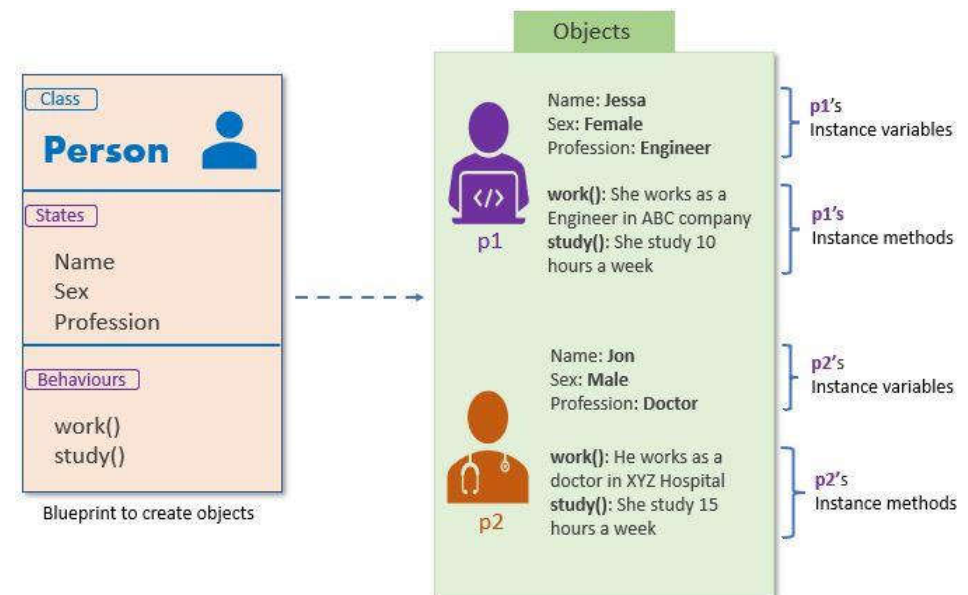
- **Self** represents the instance of the class.
- By using the “**self**” we can access the variables, attributes and methods of the class

```
In [2]: class CrawlerArticle():
        def __init__(self, title, emoji,
            self.title = title
            self.emoji = emoji
            self.content = content
            self.image = image

        class ArticleFetcher():
            def fetch(self):
                r = requests.get("http://pyth
                doc = BeautifulSoup(r.text, "
```

Object Properties.

- If we design a class based on the states and behaviors of a Person, then States can be represented as instance variables and behaviors as class methods.
 - **Identity:** Every object must be uniquely identified.
 - **State:** An object has an attribute that represents a state of an object, and it also reflects the property of an object.
 - **Behavior:** An object has methods that represent its behavior.



Example

```
1 usage
class Person:
    # Constructor
    def __init__(self, name, sex, profession):
        # State (instance variables)
        self.name = name
        self.sex = sex
        self.profession = profession

    # Behavior (instance methods)
1 usage
    def show(self):
        print('Name:', self.name, '\nSex:', self.sex, '\nProfession:', self.profession)

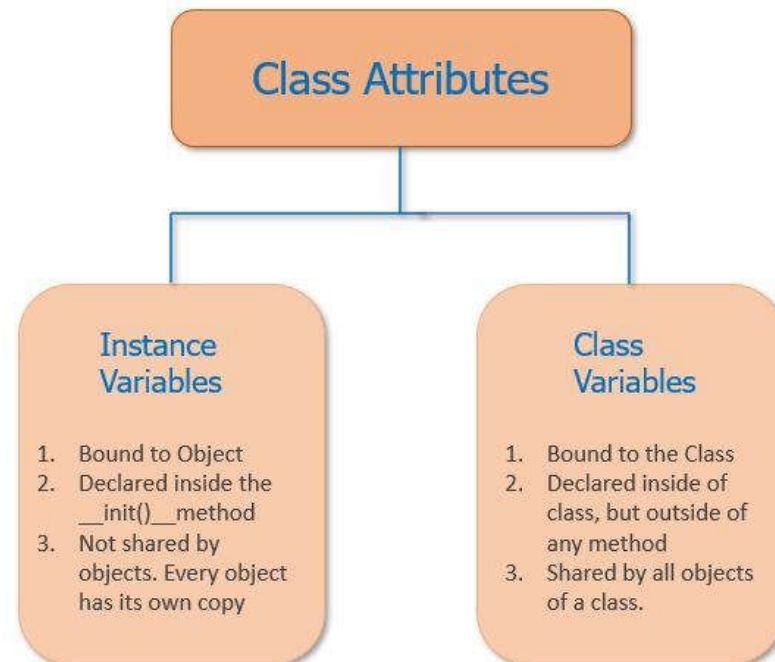
1 usage
    def work(self):
        print(self.name, 'working as a', self.profession, 'at', 'ABC Company')

# create object of a class
jessa = Person(name: 'Jessa', sex: 'Female', profession: 'Software Engineer', 10)

# call methods
jessa.show()
jessa.work()
```

Class Attributes

- Class, attributes can be defined **into two parts**:
 - Instance variables: The instance variables are attributes attached to an instance of a class. We define instance variables in the constructor (the `__init__()` method of a class).
 - Class Variables: A class variable is a variable that is declared inside of class, but outside of any instance method or `__init__()` method.

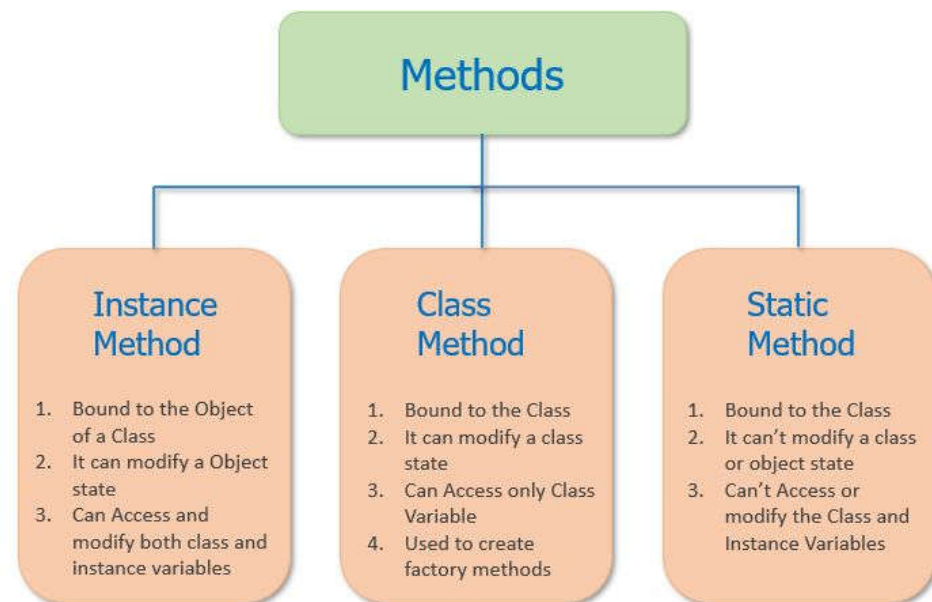


Example

```
1 class Student:
2     # class variables
3     school_name = 'ABC School'
4
5     # constructor
6     def __init__(self, name, age):
7         # instance variables
8         self.name = name
9         self.age = age
10
11 s1 = Student( name: "Harry", age: 12)
12 # access instance variables
13 print('Student:', s1.name, s1.age)
14
15 # access class variable
16 print('School name:', Student.school_name)
17
18 # Modify instance variables
19 s1.name = 'Jessa'
20 s1.age = 14
21 print('Student:', s1.name, s1.age)
22
23 # Modify class variables
24 Student.school_name = 'XYZ School'
25 print('School name:', Student.school_name)
```

Class Methods

- Inside a Class, we can define the following three types of methods.
- **Instance method**: Used to access or modify the object state. If we use instance variables inside a method, such methods are called instance methods.
- **Class method**: Used to access or modify the class state. In method implementation, if we use only class variables, then such type of methods we should declare as a class method.
- **Static method**: It is a general utility method that performs a task in isolation. Inside this method, we don't use instance or class variable because this static method doesn't have access to the class attributes.



Example

```
@usages
class Student:
    school_name = "Negros Occidental University" # Class Variable

    def __init__(self, name, age, grade):
        """Instance Method: Initializes an instance of Student."""
        self.name = name
        self.age = age
        self.grade = grade # Example: "A", "B", "C"

    1 usage
    def display_info(self):
        """Instance Method: Displays student details."""
        return f"Student Name: {self.name}, Age: {self.age}, Grade: {self.grade}"

    1 usage
    @classmethod
    def change_school(cls, new_school):
        """Class Method: Changes the school name for all students."""
        cls.school_name = new_school

    2 usages
    @staticmethod
    def is_passing(grade):
        """Static Method: Checks if the student passed."""
        return grade in ["A", "B", "C"]

# Creating Student Instances
student1 = Student(name="Alice", age=20, grade="A")
student2 = Student(name="Bob", age=22, grade="D")

# Using Instance Method
print(student1.display_info()) # Output: Student Name: Alice, Age: 20, Grade: A

# Using Class Method to change school name
Student.change_school("Silay City Institute")
print(Student.school_name) # Output: Silay City Institute

# Using Static Method
print(Student.is_passing(student1.grade)) # Output: True
print(Student.is_passing(student2.grade)) # Output: False
```

END

